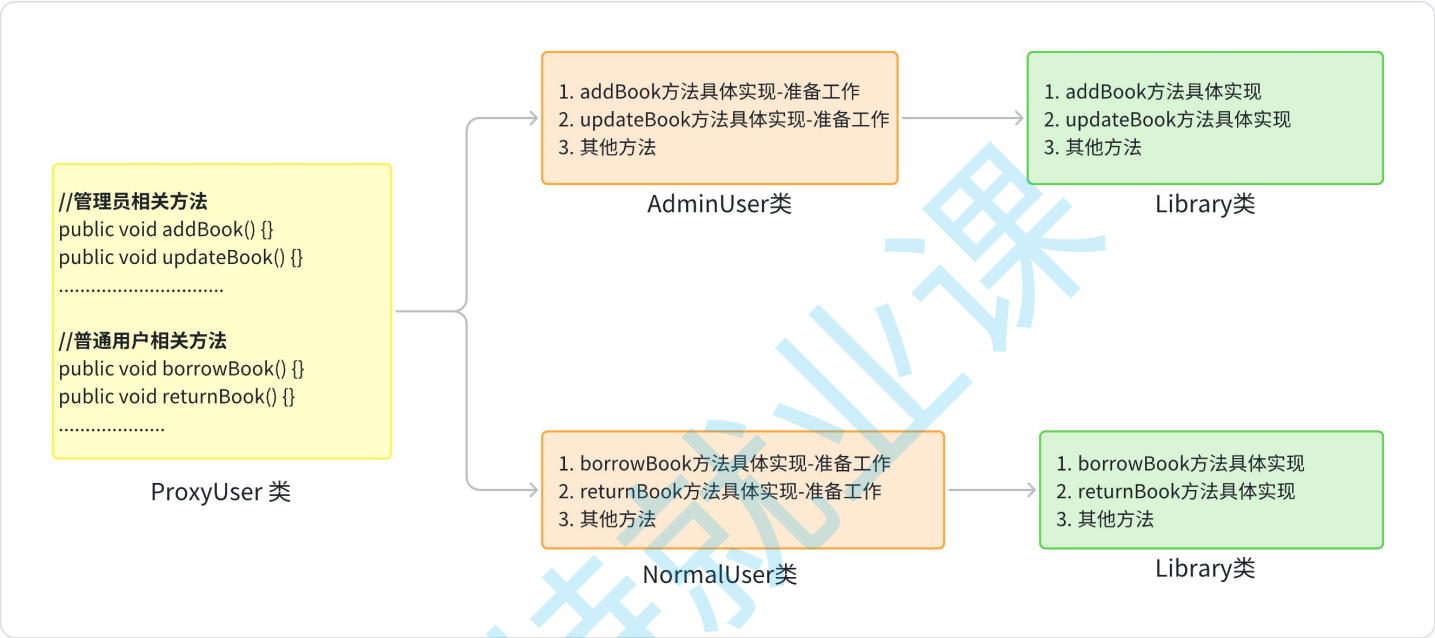


21.图书系统项目（五）

本节主要完成**管理员**业务逻辑具体实现

1. 回顾业务流程

我们前面已经分析过了，整个业务逻辑框架的实现流程如下：



2. 管理员端业务实现

2.1 添加书籍

检查权限是否合法-》调用管理员内部添加方法准备数据-》调用Libary方法进行业务具体实现



1. ProxyUser类：

```

2    * 检查当前代理用户是否为管理员
3    */
4    private void checkRealUserWhetherAdminUser(String exceptionMessage) {
5        if(!(realUser instanceof AdminUser)){
6            //自定义异常类，表示权限异常
7            throw new PermissionException(exceptionMessage);
8        }
9    }
10
11    //添加书籍操作
12    public void addBook() {
13        //查看代理的对象是不是管理员
14        checkRealUserWhetherAdminUser("普通用户没有权限上架图书");
15        ((AdminUser) realUser).addBook();
16    }

```

2. AdminUser类:

```

1    //上架图书
2    public void addBook() {
3        scanner.nextLine();
4        System.out.println("请输入书名:");
5        String title = scanner.nextLine(); // 输入书名
6        System.out.println("请输入作者:");
7        String author = scanner.nextLine(); // 输入作者
8        System.out.println("请输入类别:");
9        String category = scanner.nextLine(); // 输入图书类别
10       System.out.println("请输入出版年份:");
11       int year = scanner.nextInt(); // 输入出版年份
12       scanner.nextLine(); // 吞掉换行符
13
14       LocalDate shelfDate = LocalDate.now(); // 当前时间作为上架时间
15       Book newBook = new Book(title, author, category, year, shelfDate); // 创
建新书对象
16
17       //调用图书类 添加图书
18       library.addBook(newBook);
19   }

```

3. Library类

```

1    //添加图书
2    public void addBook(Book book) {

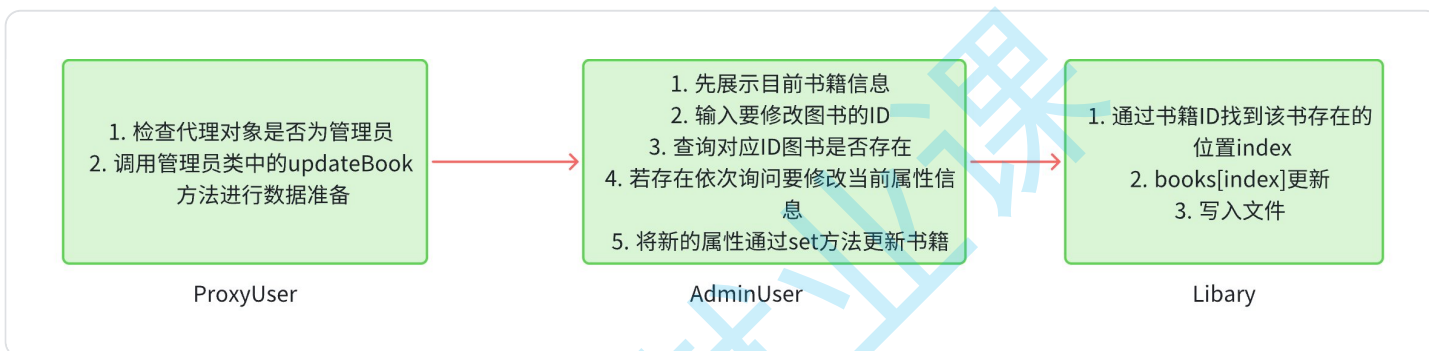
```

```

3         if(bookCount >= books.length) {
4             System.out.println("图书馆已满，无法上架更多图书！");
5             return;
6         }
7         //修改bookId为自增
8         book.setBookId(bookCount+1);
9         books[bookCount++] = book;
10        //此时存储数据的时候 会把书籍对象全部存储，虽然部分属性没有输入赋值
11        storeBook();
12        System.out.println("图书 "+book.getTitle()+"上架成功！");
13    }

```

2.2 更新书籍



1. ProxyUser类:

```

1    //更新书籍操作
2    public void updateBook() {
3        checkRealUserWhetherAdminUser("普通用户没有权限更新图书");
4        ((AdminUser) realUser).updateBook();
5    }

```

2. AdminUser类:

```

1    //图书修改 支持修改书名 作者 类别
2    public void updateBook() {
3        //1. 先展示一下目前的所有书籍
4        library.displayBooks();
5        System.out.println("请输入要修改的图书id: ");
6        int bookId = scanner.nextInt();
7        // 吞掉换行符
8        scanner.nextLine();
9        // 获取对应的图书
10       Book book = library.searchById(bookId);

```

```

11     if(book == null) {
12         System.out.println("没有ID为: "+bookId+" 的书籍! ");
13         return;
14     }
15     System.out.println("当前书名: " + book.getTitle());
16     System.out.println("请输入新的书名: ");
17     String newTitle = scanner.nextLine(); // 输入新的书名
18     System.out.println("当前作者: " + book.getAuthor());
19     System.out.println("请输入新的作者: ");
20     String newAuthor = scanner.nextLine(); // 输入新的作者
21     System.out.println("当前类别: " + book.getCategory());
22     System.out.println("请输入新的类别: ");
23     String newCategory = scanner.nextLine(); // 输入新的类别
24
25     //更新对应书籍的信息
26     book.setTitle(newTitle);
27     book.setAuthor(newAuthor);
28     book.setCategory(newCategory);
29
30     library.updateBook(book);
31 }

```

3. Library类

```

1 //更新图书
2 public void updateBook(Book book) {
3     //先找到该书是哪个下标
4     int index = searchByIdReturnIndex(book.getBookId());
5     books[index] = book;
6     //一定要进行存储
7     storeBook();
8 }
9 //根据bookId 返回 书籍索引位置
10 private int searchByIdReturnIndex(int bookId) {
11     loadAllBook();
12     for (int i = 0; i < getBookCount(); i++) {
13         Book book = books[i];
14         if(book.getBookId()==bookId) {
15             return i;
16         }
17     }
18     return -1;
19 }
20
21 //根据bookId 查找对应书籍

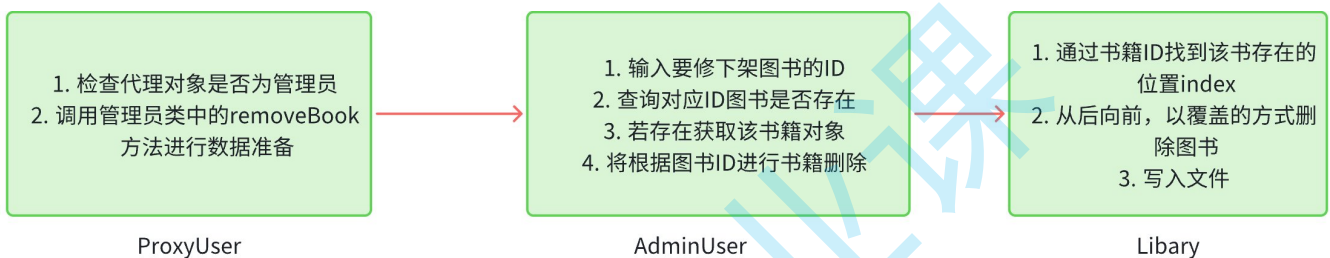
```

```

22 public Book searchById(int bookId) {
23     loadAllBook();
24     for (int i = 0; i < getBookCount(); i++) {
25         Book book = books[i];
26         if(book.getBookId()==bookId) {
27             return book;
28         }
29     }
30     return null;
31 }

```

2.3 移除(下架)书籍



1. ProxyUser类:

```

1 //移除图书
2 public void removeBook() {
3     checkRealUserWhetherAdminUser("普通用户没有权限删除图书");
4     ((AdminUser) realUser).removeBook();
5 }

```

2. AdminUser类:

```

1 //删除书籍
2 public void removeBook() {
3     //1.展示一下所有的图书
4     library.displayBooks();
5     System.out.println("请输入要删除的图书的ID: ");
6     int bookId = scanner.nextInt();
7     scanner.nextLine(); // 吞掉换行符
8 }

```

```

9      //记录一下删除的图书对象
10     Book removeBook = library.searchById(bookId);
11     //开始删除
12     library.removeBook(bookId);
13
14     System.out.println("图书: "+removeBook.getTitle()+" 已经被删除! ");
15 }

```

3. Library类

```

1  //删除图书
2  public void removeBook(int bookId) {
3
4      int index = searchByIdReturnIndex(bookId);
5      //开始删除 从当前位置的后边往前移动
6      for (int i = index; i < bookCount-1; i++) {
7          books[i] = books[i+1];
8      }
9      books[bookCount-1] = null;
10
11     storeBook();
12
13     bookCount--;
14 }

```

2.4 查看图书借阅次数



1. ProxyUser类:

```

1  //查看图书的借阅次数
2  public void borrowCount( ) {
3      checkRealUserWhetherAdminUser("普通用户没有权限查看图书的借阅次数");
4      ((AdminUser) realUser).borrowCount();

```

```
5 }
```

2. AdminUser类:

```
1 //统计每本书的借阅次数
2 public void borrowCount() {
3     library.borrowCount();
4 }
```

3. Library类

```
1 //每本书的借阅次数
2 public void borrowCount() {
3     loadAllBook();
4     for (int i = 0; i < bookCount; i++) {
5         Book book = books[i];
6         System.out.println("书名:"+book.getTitle()+
7             " 借阅次数: "+book.getBorrowCount());
8     }
9 }
```

2.5 查看最受欢迎的前K本书



1. ProxyUser类:

```
1 //查看最受欢迎的前K本书
2 public void generateBook() {
3     checkRealUserWhetherAdminUser("普通用户没有权限查看最受欢迎的前k本书");
4     ((AdminUser) realUser).generateBook();
5 }
```

2. AdminUser类:

```
1 //查询最受欢迎的前n本书
2 public void generateBook() {
3     System.out.println("请输入你要查看的最受欢迎的前K本书,注意k值不能超
过: "+library.getBookCount());
4     int k = scanner.nextInt();
5     if(k <= 0 || k > library.getBookCount()) {
6         System.out.println("没有最受欢迎的前"+k+"本书! ");
7         return;
8     }
9     library.generateBook(k);
10 }
```

3. Library类

```
1 //查询最受欢迎的前n本书
2 public void generateBook(int k) {
3     //1. 加载已有的全部的书籍
4     loadAllBook();
5     //2.把所有书籍放在 临时数据 进行排序
6     Book[] tmp = new Book[getBookCount()];
7     for (int i = 0; i < getBookCount(); i++) {
8         tmp[i] = books[i];
9     }
10    //2.1 开始排序
11    Arrays.sort(tmp);
12    //3. 把前k本书拷贝到新数组 可以不定义临时数组,直接输出前K个就行
13    Book[] generateBooks = new Book[k];
14
15    for (int i = 0; i < k; i++) {
16        generateBooks[i] = tmp[i];
17    }
18    //4.打印新数组
19    System.out.println("最受欢迎书籍如下: ");
20    for (int i = 0; i < generateBooks.length; i++) {
21        Book book = generateBooks[i];
22        System.out.println("索引: "+i+" 书名: "+ book.getTitle()+" 作者: "+
23            book.getTitle()+" 借阅次数: "+book.getBorrowCount());
24    }
25 }
```

2.6 查看库存状态



1. ProxyUser类：

```
1 //查看库存状态
2 public void checkInventoryStatus() {
3     checkRealUserWhetherAdminUser("普通用户没有权限查看库存状态");
4     ((AdminUser) realUser).checkInventoryStatus();
5 }
```

2. AdminUser类：

```
1 //查看库存状态
2 public void checkInventoryStatus() {
3     library.checkInventoryStatus();
4 }
```

3. Library类

```
1 //查看库存状态
2 public void checkInventoryStatus() {
3     loadAllBook();
4     for (int i = 0; i < bookCount; i++) {
5         Book book = books[i];
6         String status = book.isBorrowed() ? "已借出" : "在馆";
7         System.out.println("书名: "+book.getTitle()+" 目前状态: "+status);
8     }
9 }
```

2.7 移除上架超过1年的书籍



1. ProxyUser类:

```
1 //移除上架超过1年的书籍
2 public void checkAndRemoveOldBooks() {
3     checkRealUserWhetherAdminUser("普通用户没有权限移除上架超过一年的图书");
4     ((AdminUser) realUser).checkAndRemoveOldBooks();
5 }
```

2. AdminUser类:

```
1 //并移除上架超过一年的图书
2 public void checkAndRemoveOldBooks() {
3     library.checkAndRemoveOldBooks(scanner);
4 }
```

3. Library类

```
1 //移除上架超过一年的图书
2 public void checkAndRemoveOldBooks(Scanner scanner) {
3     loadAllBook();
4     // 获取当前时间戳
5     long currentTimeStamp = System.currentTimeMillis();
6
7     // 将当前时间戳转换为 LocalDate
8     LocalDate currentDate = Instant.ofEpochMilli(currentTimeStamp)
9         .atZone(ZoneId.systemDefault())
10        .toLocalDate();
11
12
13     boolean flg = false;
14     for (int i = 0; i < getBookCount(); i++) {
15         Book book = books[i];
```

```

16         //获取当前书籍的上架时间
17         LocalDate specifiedDate = book.getShelfDate();
18
19         // 计算两个日期之间的差值（以年为单位）
20         long yearsBetween = ChronoUnit.YEARS.between(specifiedDate,
currentDate);
21
22         if(yearsBetween >= 1) {
23             System.out.print("图书 " + book.getTitle() + " 已经上架超过一年，是否
移除? (y/n): ");
24             scanner.nextLine();
25             String response = scanner.nextLine();
26             if (response.equalsIgnoreCase("y")) {
27                 //确认删除调用remove方法进行删除
28                 removeBook(i);
29                 i--; // 因为后面的书已经向前移动，所以要重新检查当前索引位置
30             }
31             flg = true;
32         }
33     }
34     if(!flg) {
35         System.out.println("没有上架超过一年的图书!");
36     }
37 }

```

1. `Instant.ofEpochMilli(currentTimestamp)` 方法的作用是将毫秒级时间戳转换为 `Instant` 对象(表示时间线上的一个瞬时点)
2. 将 `Instant` 对象（UTC时间）根据系统默认时区转换为带时区的 `ZonedDateTime` 对象。
3. `toLocalDate()` 将 `ZonedDateTime` 转换为 `LocalDate` 对象